

一，背景

一个完整的大模型应用不仅包括模型本身，还包括模型部署、请求调度、推理优化、缓存管理以及服务架构设计等多个环节。当用户向大模型发送请求时，请求并不会简单地直接输入模型，而是需要经过推理服务框架进行处理，包括请求接收、任务调度、输入计算（Prefill）、输出生成（Decode）以及缓存管理等过程。

client-> tokenizer-> scheduler / queue-> prefill-> KV Cache / RadixCache->

decode iterations-> sampling-> streaming output-> metric

本挑战希望同学们通过部署并使用 SGLang 在线推理框架，亲身体验一次完整的大模型服务流程，并进一步理解现代推理系统中的关键优化技术，包括 RadixAttention、RadixCache、KV Cache、Prefix Reuse 等。同时，通过模拟真实业务请求 workload，使同学能够从单次模型调用扩展到在线服务系统层面，理解大模型推理性能优化背后的系统设计思想。

二，创建 github 账号并创建公开仓库

三，部署以及环境要求

在本地环境中启动 SGLangOpenAI-compatible 服务，能访问 /v1/models，并完成一次推理请求。

第一步：准备环境

建议 Ubuntu / WSL。

环境要求：

SGLang **0.5.14**

Ray **2.56.0**

模型 **Qwen/Qwen3-0.6B**

并且 SGLang 和 Ray 可以放在不同 Python 环境，通过 HTTP 通信。

使用要求的模型：

Qwen/Qwen3-0.6B

四，挑战内容

在本地环境中启动 SGLangOpenAI-compatible 服务，能访问 /v1/models，并完成一次推理请求。

1. 发送一次推理请求并截图保存（截图 1）

2. Mooncake trace 采样 workload

从 Mooncake FAST'25 trace 中采样 10-30 条请求记录，例如 arxiv-trace/mooncake_trace.jsonl。根据 input_length 和 output_length 构造 synthetic

prompt, 用 Poisson 到达过程生成请求时间, 并将这组小 workload 发送到 SGLang。记录每个请求的 input tokens、output tokens、status、TTFT 或 latency。截图保存 (截图 2)

3. 用一张流程图讲清一次请求如何经过 SGLang 在线推理框架, 重点放在流程上

4. 重点回答:

- SGLang 的 RadixAttention / RadixCache 解决什么问题?
- SGLang 中 page-sized KV cache 与 prefix reuse 分别对应 pipeline 中的哪一部分?
- vLLM 的 PagedAttention 解决什么问题? 它和 SGLang 的 RadixAttention / paged KV 机制有什么联系与差异?

答案文件不超过一页

五, 作业感受

(1) 简述下你是如何完成第一次挑战的。(例如, 如何使用 AI、如何工作迭代等方面)

(2) 你觉得哪部分工作最困难, 你是如何克服这个困难的。

(3) 你觉得类似的从 0 到 1 的科研工作, 需要包含哪些方面的研究, 对你今后的科研工作有什么启发

六, AI 使用说明情况

使用的 AI 模型, 使用的 AI 提示词, AI 如何帮助你学习, 你在学习哪方面的时候使用 AI 以及在运用 AI 时是否被误导

七, 最终交付

Github 仓库链接, 仓库包含:

- 1.将截图 1, 2 放在同一个 pdf 文件, 命名为操作保存.pdf
- 2.流程图.pdf
- 3.重点回答.pdf
- 4.作业感受.pdf
5. AI 使用说明情况.pdf
- 6.阅读文献笔记.pdf

截止时间为 9.27 22: 00

提交方式: 在群里面的收集表中提交 github 仓库链接

八. 参考

SGLang Documentation: <https://docs.sglang.ai/>

- SGLang / RadixAttention Paper (NeurIPS 2024) :
https://papers.nips.cc/paper_files/paper/2024/hash/724be4472168f31ba1c9ac630f15dec8-Abstract-Conference.html
- vLLM Documentation: <https://docs.vllm.ai/>
- vLLM / PagedAttention Paper (SOSP 2023) :
<https://dl.acm.org/doi/10.1145/3600006.3613165>
- Mooncake FAST'25 Trace Release: <https://github.com/kvcache-ai/Mooncake/tree/main/FAST25-releas>